
Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2022

1. Thread activo [2 puntos]

Un sistema tiene NTHREADS threads, de los que únicamente hay uno activo en cada momento. Ese thread le pasa el turno a otro thread del grupo, e imprime un mensaje con su número y el del thread que continúa el ciclo.

```
#define NTHREADS ...

struct tag_group {
    int current_thread; // active thread
    ...
};

struct thr_args {
    int my_number;
    struct tag_group *t; // Shared
};

void thread(void *ptr) {
    struct thr_args *p = ptr;
    int next;

    while(1) {
        ...
        next = rand() % NTHREADS;
        printf("thread %d tagging thread %d\n", p->my_number, next);
        ...
    }
}
```

Implemente la función thread de tal forma que el comportamiento sea el descrito. Cada thread debe despertar únicamente al siguiente, no a todos.

```
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*fun) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Solución

```
#define NTHREADS ...

struct tag_group {
    int current_thread; // active thread
    pthread_mutex_t m;
    pthread_cond_t cond[NTHREADS];
};

struct thr_args {
    int my_number;
    struct tag_group *t; // Shared
};

void thread(void *ptr) {
    struct thr_args *args = ptr;
    int next;

    while(1) {
        pthread_mutex_lock(&args->t->m);
        while(args->t->current_thread != args->my_number)
            pthread_cond_wait(&args->t->cond[args->my_number], &args->t->m);
        pthread_mutex_unlock(&args->t->m);

        next = rand() % NTHREADS;
        printf("thread %d tagging thread %d\n", args->my_number, next);

        pthread_mutex_lock(&args->t->m);
        args->t->current_thread = next;
        pthread_cond_signal(args->t->cond[next]);
        pthread_mutex_unlock(&args->t->m);
    }
}
```

2. Sistema de gestión de memoria [2 puntos]

Implemente un sistema de gestión de memoria. El API tiene dos operaciones, `malloc()` y `free()`. Existen varios buckets de tamaños de memoria, todos potencias de 2: 8, 16, 32, 64, 128, 256, 512. El sistema funciona de la siguiente manera:

- Existe un array `buckets` que contiene una entrada para cada uno de los tamaños.
- Cada entrada de este array tiene una lista de bloques libres del tamaño correspondiente.
- Cada elemento de la lista está compuesto por un campo de la estructura `elem`, que contiene un puntero a la zona de memoria libre `pointer`.
- La función `elem_create()` crea un nuevo elemento.
- La función `elem_delete(elem)` borra el elemento `elem`.
- La función `list_insert(list, elem)` inserta el elemento `elem` en la lista `list`.
- La función `list_next()` devuelve el primer elemento de la lista, si la lista esta vacía devuelve `NULL`. Este elemento se borra de la lista.
- Las funciones anteriores no hay que implementarlas.

Hay que implementar las siguientes funciones:

- `void *malloc(size_t size)`. Esta función busca un bloque de memoria en el primer bucket en el que cabe este trozo. Es decir `malloc(7)` iría en el bucket de 8, y `malloc(59)` en el de 64.
- Si no hay buckets disponibles para el tamaño correspondiente, se busca un bucket en el tamaño inmediatamente superior, se parte en dos, uno se mete en la lista y el otro se devuelve.
- `void free(void *pointer, size_t size)`. Esta función libera el bloque de memoria que se le pasa. Nos indica el tamaño del bloque que se ha asignado para hacer todo más sencillo.
- El sistema tiene que funcionar de forma multithread. Es decir, puede haber varios `malloc()` y `free()` ejecutándose al mismo tiempo.
- Hay que minimizar el tiempo que hay un mutex bloqueado. Pista: Úsese un mutex por bucket.
- El array de buckets está indexado por la potencia de 2 que contiene, teniendo en cuenta que empezamos en tamaño 8. La posición 0 contiene el tamaño 8, la posición 2 contiene el tamaño 16, etc. Es decir, el tamaño n (donde $n = 2^c$) se almacena en la posición $c - 3$.

```
struct list_t;

struct elem {
    pointer *pointer; /* pointer to memory */
};

struct bucket {
    size_t size;
    list_t *blocks;
    ...
};
struct bucket buckets[NUM];

struct elem *elem_create(void);
void elem_delete(struct elem *elem);
void list_insert(struct list_t list, struct list_t *elem);
struct elem *list_next(struct list_t *list,)
```

Solución

```
struct list_t;

struct elem {
    pointer *pointer; /* pointer to memory */
};

struct bucket {
    size_t size;
    list_t *blocks;
    pthread_mutex_t *m;
};

struct bucket buckets[NUM];

struct elem *elem_create(void);
void elem_delete(struct elem *elem);
void list_insert(struct list_t list, struct list_t *elem);
struct elem *list_next(struct list_t *list)

// Power of the sizes
#define MIN_SIZE 3
#define MAX_SIZE 9

struct elem *buddy_malloc(int i)
{
    if (i >= MAX_SIZE)
        return NULL;
    pthread_mutex_lock(buckets[i - MIN].m);
    struct elem *e = list_next(buckets[i - MIN].blocks);
    pthread_mutex_unlock(buckets[i - MIN].m);
    if (e != NULL)
        return e;
    e = buddy_malloc(i + 1);
    if (e == NULL)
        return NULL;
    struct elem *buddy = elem_create();
    pthread_mutex_lock(buckets[i - MIN].m);
    list_insert(buckets[i - MIN].blocks, buddy);
    pthread_mutex_unlock(buckets[i - MIN].m);
    return e;
}

void *malloc(size_t size)
{
    for (int i = MIN_SIZE; i <= MAX_SIZE; i++) {
        if (size < 2 ^ i) {
            pthread_mutex_lock(buckets[i - MIN].m);
            struct elem *e = list_next(buckets[i - MIN].blocks);
            pthread_mutex_unlock(buckets[i - MIN].m);
            if (e == NULL) {
                return NULL;
            }
            void *p = e->pointer;
            free(e);
            return pointer;
        }
    }
    printf("Size too big %d\n", size);
}

void free(void *pointer, size_t size)
{
    for (int i = MIN_SIZE; i <= MAX_SIZE; i++) {
        if (size < 2 ^ i) {
            struct elem *e = elem_create();
            e->pointer = pointer;
            pthread_mutex_lock(buckets[i - MIN].m);
            list_insert(buckets[i - MIN].blocks, e);
            pthread_mutex_unlock(buckets[i - MIN].m);
            return;
        }
    }
    printf("Invalid size %d\n", size);
}
```

3. Sistema de log [1 punto]

En un sistema existe un servicio de log que recibe mensajes de otros procesos con un nivel de gravedad asociado, y los imprime por pantalla o no según el nivel mínimo de gravedad actual.

Para eso, el servidor proporciona un API con dos funciones:

- `set_log_level/2`, que permite cambiar el nivel de gravedad del servidor de log. Los mensajes solo se imprimirán si su nivel de gravedad es igual o superior al valor establecido.
- `log/3`, que envía un mensaje con su nivel de gravedad al servidor.

Implemente las funciones `set_log_level/2`, `log/3` y `loop` para que el sistema tenga el comportamiento descrito. Puede modificar el número de parámetros de `loop` si lo necesita.

```
-module(log).  
  
-export([start/0, init/0, set_log_level/2, log/3]).  
  
start() ->  
    spawn(?MODULE, init, []).  
  
set_log_level(Log_Server, Level) ->  
    ...  
  
log(Log_Server, Msg, Level) ->  
    ...  
  
init() ->  
    loop(0).  
  
loop(Current_Level) ->  
    ...  
  
log_message(Msg, Level) ->  
    io:format("[~w]: ~w~n", [Level, Msg]).
```

Solución

```
-module(log).  
  
-export([start/0, init/0, set_log_level/2, log/3]).  
  
start() ->  
    spawn(?MODULE, init, []).  
  
set_log_level(Log_Server, Level) ->  
    Log_Server ! {set_log_level, Level}.  
  
log(Log_Server, Msg, Level) ->  
    Log_Server ! {log, Msg, Level}.  
  
init() ->  
    loop(0).  
  
loop(Current_Level) ->  
    receive  
        {set_log_level, Level} ->  
            loop(Level);  
        {log, Msg, Level} ->  
            if  
                Level >= Current_Level ->  
                    log_message(Msg, Level);  
            true ->  
                ok  
            end,  
            loop(Current_Level)  
    end.  
  
log_message(Msg, Level) ->  
    io:format("[~w]: ~w~n", [Level, Msg]).
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria extraordinaria

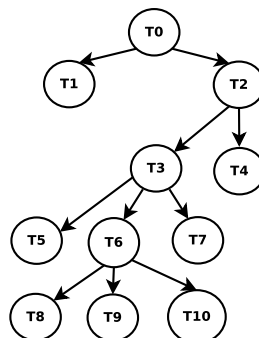
11 de julio de 2022

AVISO: No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregan por separado.

- [2,5p] 1. **Conceptos de paralelismo.** Una empresa ha desarrollado un conjunto de aplicaciones bioinformáticas pensadas para ser ejecutadas en forma de grafo. En todas las ejecuciones se empieza leyendo un genoma (cadena de caracteres A,C,G,T) de un fichero de texto y luego se pueden ejecutar varias aplicaciones sobre él, algunas de las cuales pueden ser realizadas de forma simultánea y otras no (debido a tener dependencias entre ellas). Para cada ejecución será el usuario el que elegirá qué aplicaciones se ejecutan y cuáles no, generando un grafo de tareas concreto para sus análisis.

Contesta DE FORMA RAZONADA las siguientes preguntas:

- (a) ¿Cuál es la aceleración máxima que se puede obtener si la primera tarea (lectura del genoma desde fichero de texto) siempre la hace un único proceso, no se puede simultanear con ninguna otra tarea (nada puede empezar hasta que el genoma esté en memoria) y siempre tarda el 5% del tiempo total?[0,5p]
- (b) Un biólogo utiliza un grafo con 11 tareas, con las dependencias representadas en la siguiente figura y, cada una, requiere los siguientes tiempos para un determinado genoma:



- Tareas 0, 4, 9 y 10: 10 minutos.
- Tareas 1, 3, 5, 7 y 8: 20 minutos.
- Tareas 2 y 6: 30 minutos.

¿Cuáles son el Máximo Grado de Concurrencia y el Grado Medio de Concurrencia? [0,5p]

- (c) En el caso de disponer de tres procesos, ¿cuál sería la asignación óptima? [0,5p]
- (d) ¿Cuáles son el speedup y la eficiencia de la asignación propuesta en el apartado anterior? [0,5p]
- (e) ¿Se consigue alguna ganancia real de tiempo al usar tres procesos, comparado con usar solo dos? Y si tuviésemos más de tres procesos disponibles, ¿sería interesante usar más? [0,5p]

- [2,5p] 2. **Diseño de algoritmos paralelos.** Una de las aplicaciones de la librería presentada en el ejercicio anterior consiste en comparar dos secuencias de ADN (cadenas de caracteres) y proporcionar como resultado el número de posiciones en las que ambas secuencias tienen el mismo valor. A continuación, presentamos el código de la parte principal de dicha aplicación, que recibe como parámetro de entrada la longitud L de las secuencias (ambas deben tener la misma longitud).

```
char *seq1 = (char *)malloc(sizeof(char) * L);
char *seq2 = (char *)malloc(sizeof(char) * L);
leeDeDisco(seq1, seq2, L);
int numIgual = 0;

for(int i = 0; i < L; i++)
    if(seq1[i] == seq2[i])
        numIgual++;

escribeResultado(numIgual);
```

Como estas secuencias pueden ser muy grandes, se quiere paralelizar esa aplicación concreta, teniendo en cuenta que solo uno de los procesos tiene acceso a disco y pantalla. Por tanto, este será el único proceso que puede ejecutar las funciones `leeDeDisco` y `escribeResultado`. Contesta DE FORMA RAZONADA las siguientes preguntas:

- (a) Indica el tipo de descomposición y asignación que usarías para paralelizar esta aplicación. [0,5p]
- (b) Escribe el código MPI asociado a la solución que has indicado en el apartado anterior, usando colectivas siempre que se pueda (ver la tabla del final con la sintaxis de las rutinas MPI). Asume que L es múltiplo del número de procesos. [0,75p]
- (c) Asume que los procesos se encuentran conectados por una red en topología de anillo, donde cada proceso puede comunicarse directamente con otros dos (por ejemplo, el Proceso 1 está conectado con el 0 y con el 2). Estas conexiones son bidireccionales. Explica, DE PALABRA, cómo realizarías una colectiva `MPI_Bcast` en dicha red si tiene como proceso raíz el 0. ¿Cuántos pasos se necesitarían para completar el *broadcast* sobre sobre $numP$ procesos? [0,5p]
- (d) Escribe con código MPI (usando funciones `MPI_Send` y `MPI_Recv`) la implementación de la colectiva del apartado anterior cuando recibe como entrada un array de caracteres A de un tamaño arbitrario L . [0,75]


```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,
                 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
              MPI_Datatype datatype, MPI_Op op,
              int root, MPI_Comm comm)
```

SOLUCIÓN

1. (a) La Ley de Amdahl la aceleración máxima es:

$$\frac{T_s + T_p}{T_s}$$

En este caso, la parte intrínsecamente secuencial es el 0.05 del tiempo. Lo que quiere decir que la aceleración máxima es:

$$\frac{0.05 + 0.95}{0.05} = 20$$

- (b) El Máximo Grado de Concurrencia sería 7, ya que todos los extremos del final del árbol se podrían ejecutar a la vez.

Para calcular el Grado Medio de Concurrencia necesitamos determinar el camino crítico: T0-T2-T3-T6-T8 (longitud 110 minutos). Todas las tareas en secuencial tardan 200 minutos, así que el Grado Medio de Concurrencia es:

$$\frac{200}{110} = 1.82$$

- (c) Podría haber varias asignaciones óptimas. Una con la que el tiempo paralelo es igual que el Grado Medio de Concurrencia (y, por tanto, no es mejorable) sería:

- Proceso 0: T0, T2, T3, T6, T8
- Proceso 1: T1, T4, T5, T9
- Proceso 2: T7, T10

En la siguiente tabla se puede ver que esta asignación necesita 110 minutos (cada fragmento en horizontal representa 10 minutos):

Proceso 0	T0	T2	T2	T2	T3	T3	T6	T6	T6	T8	T8
Proceso 1		T1	T1		T4		T5	T5		T9	
Proceso 2							T7	T7		T10	

- (d) El tiempo paralelo se computa desde que se inicia el primer proceso hasta que termina el último. En este caso es la carga del Proceso 0 (110 minutos).

$$S = \frac{200}{110} = 1.82$$

$$Eff = \frac{1.82}{3} = 0.61$$

- (e) Si no tuviésemos el tercer proceso no podríamos trabajar en paralelo a la vez con T5, T6 y T7, así que sí compensa utilizarlo. Sin embargo, por culpa de las dependencias, aunque dispongamos de más procesos no hay manera de meter una cuarta fase en paralelo en ningún punto del pipeline.
2. (a) Sería una descomposición de dominio con una tarea por cada posición de las secuencias ya que en todas ellas se realiza la misma operación (comparar si son iguales y, si es necesario, sumar uno al contador). La asignación más óptima sería una estática ya que la carga de trabajo por tarea es homogénea. Daría igual una cíclica o una bloque pero, por ser más sencilla de implementar, lo lógico es usar una bloque.

```

char *seq1, *seq2;
int numIgual, resParcial;

if(rank == 0){
    seq1 = (char *)malloc(sizeof(char) * L);
    seq2 = (char *)malloc(sizeof(char) * L);
    leeDeDisco(seq1, seq2, L);
} else {
    seq1 = (char *)malloc(sizeof(char) * L/numP);
    seq2 = (char *)malloc(sizeof(char) * L/numP);
}

resParcial = 0;

MPI_Scatter(seq1, L/numP, MPI_CHAR,
    rank ? seq1 : MPI_IN_PLACE, L/numP, MPI_CHAR, 0, MPI_COMM_WORLD);
MPI_Scatter(seq2, L/numP, MPI_CHAR,
    rank ? seq2 : MPI_IN_PLACE, L/numP, MPI_CHAR, 0, MPI_COMM_WORLD);

for(int i = 0; i < L/numP; i++)
    if(seq1[i] == seq2[i])
        resParcial++;

MPI_Reduce(&resParcial, &numIgual, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

if(rank == 0)
    escribeResultado(numIgual);

```

- (c) Inicialmente el array estaría en la memoria del Proceso 0. Como tiene dos uniones bidireccionales aprovechará para mandar a la vez el array por el camino del Proceso 1 y por el camino del proceso $numP - 1$. Cada proceso, una vez que recibe, pasa el array al siguiente hasta que todos lo tengan. Hay que tener en cuenta que uno de los caminos puede ser más largo que el otro según si el número de procesos es múltiplo de dos o no.

El número de pasos será $\frac{numP}{2}$.

```

int soyFinal = 0;
if((rank == numP/2) || (rank == numP/2+1))
    soyFinal = 1;

int caminoAscendente = 0;
if(rank <= numP/2)
    caminoAscendente = 1;

if(rank == 0) {
    MPI_Send(A, L, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    MPI_Send(A, L, MPI_CHAR, numP-1, 0, MPI_COMM_WORLD);
} else {
    MPI_Recv(A, L, MPI_CHAR, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, null);
    if(soyFinal == 0){
        if(caminoAscendente == 1){
            MPI_Send(A, L, MPI_CHAR, rank+1, 0, MPI_COMM_WORLD);
        } else {
            MPI_Send(A, L, MPI_CHAR, rank-1, 0, MPI_COMM_WORLD);
        }
    }
}
}

```